CONTENT POOL · ALL SLIDES

Playwright
Beyond the Happy Path

Every slide for the talk. A talk deck takes the slides that it needs. Sven Hennessen · vensas GmbH

vensas

About me



- **Almost 20 years** of software development and architecture
- **Co-founder and co-managing director** of vensas GmbH, founded in 2023
- Consulting and development of **business software** for medium-sized companies
- Away from the computer — **hiking**, **cycling** or the **racing simulator**

Sven Hennessen
vensas GmbH

The demo app: Deploy or Die

A deployment game. Each deployment has a 50% success rate. After a deployment you must wait 30 seconds.

Frontend	React 19, PrimeReact, Rspack
Backend	ASP.NET Core minimal API on .NET 10, PostgreSQL 16
Tests	Playwright 1.63 in TypeScript, 1.62 in C#

Demo

```
aspire run
```

Let's look at the application

Locators and auto-wait

tests/locators.spec.ts

```ts
// Use this order: role, label, text. These stay correct when the
// CSS classes or the DOM structure change.
await expect(page.getByRole('heading', { name: /Deploy or Die/i })).toBeVisible();
await expect(page.getByLabel(/Who dares to deploy/i)).toBeEditable();

// The API answer comes late, but the test does not wait manually.
await page.getByRole('button', { name: /Deploy Now/i }).click();
await expect(page.getByText(/Deployment successful/i)).toBeVisible();

// New in 1.63: keep only the elements that the user sees.
const visibleButtons = page.getByRole('button').visible();
```

A locator says WHAT you want. Playwright finds the element again for each action.

Demo

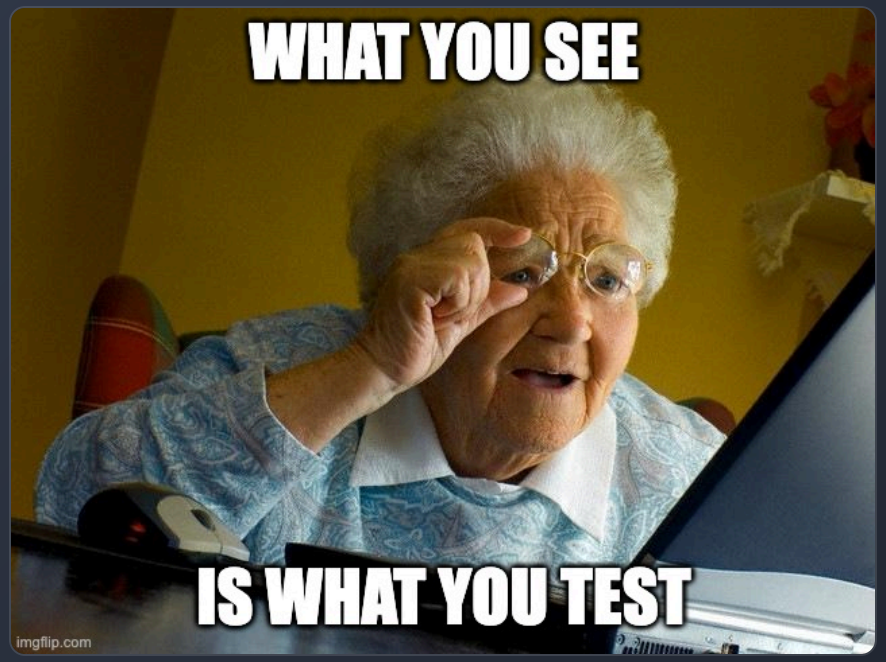
```
pnpm run test:mocked locators
```

Delete an await to see the failure message.

UI mode

What you get

- Watch mode — the test runs again after each change
- Time travel — see the DOM at each step
- Pick locator — click an element, get the locator
- Network, console and source in one window



Demo

```
pnpm run test:ui
```

FROM A REAL PROJECT

Keep the tests in their own folder

playwright.config.ts

```
// We did this. Playwright then read all of src/ at each start.
testDir: './src',

// Keep the default. The tests have their own folder.
testDir: './tests',
```

We made this change. Then we made it again in the other direction.

What it cost us

CI	The test job became slow, then it stopped with a timeout.
UI mode	The start took some minutes, but not each time.

Recommendation

Keep the tests out of the source folder.

Codegen

```
npx playwright codegen http://localhost:3000
```

Codegen records your clicks and writes the test. Since 1.55 it also writes assertions.

generated

```
// Codegen gives you a start, not a finished test.
await page.getByRole('textbox', { name: 'Who dares to deploy?' }).click();
await page.getByRole('textbox', { name: 'Who dares to deploy?' }).fill('Sven');
await page.getByRole('button', { name: 'Deploy Now 🚀 ' }).click();
await expect(page.getByText('Deployment successful')).toBeVisible();
```

Demo and warning

npx playwright codegen http://localhost:3000 — record one deployment. Then correct the hard coded name and add the negative case.

Traces

playwright.config.ts

```
use: {
  trace: {
    mode: 'on',
    snapshots: { dom: true, aria: true, screen: true },
  },
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
}
```

New in 1.63: the trace also records the ARIA tree and the screen.

A trace contains

- DOM, ARIA tree and screen for each step
- All network requests and answers
- Console output and source code

Demo

```
npx playwright show-trace test-results/.../trace.zip
```

FROM A REAL PROJECT

Make the trace available in CI

A test can fail in CI and pass on your machine. Without the trace you can only guess.

.github/workflows/e2e.yml

```yaml
- name: Upload the report
  uses: actions/upload-artifact@v7
  if: always()
  with:
    name: playwright-report
    path: playwright-report/
    retention-days: 7

- name: Upload traces and videos
  uses: actions/upload-artifact@v7
  if: failure()
  with:
    name: playwright-traces
    path: test-results/
    retention-days: 7
```

The report goes up after each run. The trace and the video go up only after a failure.

Recommendation

Upload the report after each run, and the traces after each failure.

FROM A REAL PROJECT

Keep the test pyramid

UI mode and agents make an E2E test look cheap. It is not: it costs more to write and to maintain than a unit test.

From a team that I mentored

Most tests were Playwright tests. React Testing Library was sufficient for 80% of them. The suite grew, and the CI runners became unreliable.

What I take from this

Many — **Unit and component tests. They are fast and cheap.**

Few — **E2E tests. Use them for the important paths.**

> An unreliable E2E test is worse than no E2E test.

Recommendation

Find the E2E tests that a unit or a component test can replace.

Mock the API

tests/deploy-failure.spec.ts

```
// Set the route before you go to the page.
await page.route('**/api/deploy', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({
      success: false,
      message: '💥 BOOM! The database just went on vacation.',
      deployedBy: 'Unlucky Developer',
    }),
  });
});

await page.goto('/');
```

You control the answer. Thus you can test the error path, which occurs in only 50% of the runs.

Demo

`pnpm run test:mocked`

19 tests in about 10 seconds, with no backend and no database.

The real backend

tests/testcontainers.ts

```
const composeEnvironment = await new DockerComposeEnvironment(
  composeFilePath,
  'docker-compose.integration.yml',
).up();

const backend = composeEnvironment.getContainer('backend-1');
process.env.API_URL = `http://localhost:${backend.getMappedPort(5000)}`;
```

The global setup starts the whole environment one time, before the first test.

Mock or real

Mock	**Fast and stable. Blind to the backend.**
Real	**Finds integration faults. Slow.**

Demo

```
pnpm run test:integration
```

The containers start, then 4 tests run against the real API.

FROM A REAL PROJECT

Mock or real — how I choose

Both are correct, and both add value. The complexity of the project decides.

Three questions before I decide

Services	How many services does the scenario touch?
Contract	How complex is the API contract?
Test	How complex is the test itself?

Mock or real

Mock	**Fast and stable. Blind to the backend.**
Real	**Finds integration faults. Slow.**



Recommendation

The demo app has one service and a small contract. Thus it has 19 mocked tests and 4 integration tests.

FROM A REAL PROJECT

Start the dependencies with Compose

First I started each container from the test code. It was flaky, on my machine and in CI.

What broke

- The start order and the readiness check
- The network between the containers and the port mapping
- Containers that stayed after the run

tests/docker-compose.integration.yml

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
backend:
  depends_on:
    postgres:
      condition: service_healthy
```

depends_on with a healthcheck replaces the wait code in the test.

Recommendation

Start the containers one time in a global setup, not for each test.

Test locks — new in 1.63

All integration tests write to the same database. Before 1.63 you made the whole file serial, or you accepted the flakiness.

tests/deploy-integration.spec.ts

```
test.describe('Integration Tests', { lock: 'deploy-db' }, () => {
  test('should deploy against the real backend', async ({ page }) => {
    // ...
  });

  test('the API rejects a request without a name', async ({ request }) => {
    // ...
  });
});

// The other tests keep their parallel speed.
```

Two tests with the same lock name never run at the same time — across files and across workers. Playwright Test only — the .NET bindings have no lock, because xUnit is the runner there.

Demo

Run the integration project with `--workers=4`.

The Timeline in the report shows that the four locked tests do not overlap.

Speedboard and Timeline

The mocked tests need 10 seconds. The integration tests need more than 4 minutes. The report shows you where the time goes.

```
pnpm run test:report
```

Two tabs in the HTML report

Speedboard	A list of all tests, sorted by duration. The slowest test is at the top.
Timeline	The wall clock time of the run. It shows where a worker waits.

> Most of our 4 minutes is the container build, not the tests.

Demo

```
pnpm run test:report
```

After the integration run — look at Speedboard, then at Timeline.

Isolated retries — new in 1.62

playwright.config.ts

```ts
export default defineConfig({
  retries: process.env.CI ? 2 : 0,

  // New in 1.62.
  retryStrategy: 'isolated',
});
```

The retries run one after the other at the end of the run, not immediately under full load. Playwright Test only — not available in the .NET bindings.

Why this helps

A test that fails only because of parallel load now passes on the isolated retry. This tells you the cause: it is the load, not the code.

Demo

`pnpm run test:retries`

The red test retries. With `isolated` both retries run last, back to back. Comment the line out and run again: the first retry then runs in the middle of the suite.

FROM A REAL PROJECT

The E2E tests are not only a frontend job

Teams put the E2E tests in the frontend pipeline only. But a test against a real backend also tests the backend.

What happened to a team that I mentored

The team released an API that was not compatible. Nobody ran the E2E tests before the release.

What I recommend

- Run the tests after each frontend build
- Run them after each backend build too
- Or run them nightly — your test plan decides

Recommendation

Find out where your E2E tests run today.

More than one page at a time

tests/multiple-pages.spec.ts

```
test('two users deploy at the same time', async ({ browser }) => {
  const alice = await browser.newContext();
  const bob = await browser.newContext();
  await Promise.all([mockDeploy(alice), mockDeploy(bob)]);

  const [alicePage, bobPage] = await Promise.all([alice.newPage(), bob.newPage()]);
  await Promise.all([alicePage.goto('/'), bobPage.goto('/')]);

  await Promise.all([deploy(alicePage, 'Alice'), deploy(bobPage, 'Bob')]);

  await expect(alicePage.getByText(/Deployed by: Alice/i)).toBeVisible();
  await expect(bobPage.getByText(/Deployed by: Bob/i)).toBeVisible();
});
```

Each context has its own cookies and storage. Promise.all lets both users click at the same time.

Pages and contexts

Page	One tab. context.newPage()
Context	One user. browser.newContext() — the routes apply to all its pages
New tab	context.waitForEvent('page') — start to wait before the click

Demo

```
pnpm run test:mocked multiple-pages
```

The Clock API

The app blocks a second deployment for 30 seconds. A test must not wait 30 seconds.

tests/cooldown.spec.ts

```ts
await page.clock.install();
await page.goto('/');

await page.getByRole('button', { name: /Deploy Now/i }).click();
await expect(page.getByRole('button', { name: /Next deploy in 30s/i })).toBeVisible();

// Move the time forward by 5 seconds and let the timers run.
await page.clock.runFor(5_000);
await expect(page.getByRole('button', { name: /Next deploy in 25s/i })).toBeVisible();

// Skip the full cooldown. The test needs milliseconds, not 30 seconds.
await page.clock.fastForward('00:30');
await expect(page.getByRole('button', { name: /Deploy Now/i })).toBeEnabled();
```

install() must come before every other clock call.

Demo

```
pnpm run test:clock
```

3 tests in about 2 seconds, for a cooldown of 30 seconds.

Accessibility as a gate

tests/accessibility.spec.ts

```
const results = await new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa', 'wcag22aa'])
  .analyze();

await testInfo.attach('axe-results.json', {
  body: JSON.stringify(results.violations, null, 2),
  contentType: 'application/json',
});

expect(results.violations.map((v) => ({ rule: v.id, impact: v.impact }))).toEqual([]);
```

The European Accessibility Act applies since June 2025. Thus this check belongs in the pipeline.

Expectation management

axe-core finds 25% to 57% of the defects*.

Recommendation: test the rest by hand, with the keyboard and a screen reader.

Demo

`pnpm run test:a11y`

Fails with button-name, color-contrast and label. Set `ACCESSIBLE_MODE = true` in `src/a11yMode.ts` and it passes.

* Deque, 57% · UK Government Digital Service tool audit, 30–40% · Karl Groves, 25–33%

ARIA snapshots

tests/accessibility.spec.ts

```
await expect(page.locator('.card-header')).toMatchAriaSnapshot(`
  - heading /Deploy or Die/ [level=1]
  - paragraph: /Every deployment is a gamble/
`);
```

The snapshot asserts the accessibility tree, not the DOM. It shows roles and names, not CSS.

Why this is different from axe-core

axe-core	Finds known WCAG defects, for example too little contrast
ARIA snapshot	Finds a change of the structure, for example a lost role or a wrong heading level

> The two checks find different faults. Use both.

Demo

Change the `h1` to an `h2`.

The axe test still passes, but the ARIA snapshot fails.

The same test in C#

DeployTests.cs

```
public class DeployTests : PageTest
{
    [Fact]
    public async Task Cooldown_BlocksTheSecondDeployment()
    {
        await Page.Clock.InstallAsync();
        await Page.GotoAsync("http://localhost:3000");

        var deploy = Page.GetByRole(AriaRole.Button,
            new() { NameRegex = new Regex("Deploy Now") });
        await deploy.ClickAsync();

        // Move the time forward instead of a wait of 30 seconds.
        await Page.Clock.FastForwardAsync("00:30");
        await Expect(deploy).ToBeEnabledAsync();
    }
}
```

The API is equal in all languages. Only the names use the .NET style. The package is Microsoft.Playwright.Xunit.v3 — NUnit and MSTest also exist.

Demo

```
aspire run
dotnet test
```

4 tests pass.

The same test in C#

DeployTests.cs

```
public class DeployTests : PageTest
{
    [Fact]
    public async Task Cooldown_BlocksTheSecondDeployment()
    {
        await Page.Clock.InstallAsync();
        await Page.GotoAsync("http://localhost:3000");

        var deploy = Page.GetByRole(AriaRole.Button,
            new() { NameRegex = new Regex("Deploy Now") });
        await deploy.ClickAsync();

        // Move the time forward instead of a wait of 30 seconds.
        await Page.Clock.FastForwardAsync("00:30");
        await Expect(deploy).ToBeEnabledAsync();
    }
}
```

The same API, with .NET names. Your test runner is xUnit, NUnit or MSTest.

Same	**Locators, Clock API, routes, traces, codegen, axe-core**
Node only	**UI mode, test locks, isolated retries, Test Agents**

What the .NET bindings give you

The browser API is the same. The test runner is not — in .NET your runner is xUnit, NUnit or MSTest, not Playwright Test.

Works in C#

Locators	GetByRole, GetByLabel, auto-wait
Clock API	InstallAsync, FastForwardAsync
ARIA snapshots	ToMatchAriaSnapshotAsync
Mocking	RouteAsync and the API request context
Traces	Recorded, then read with show-trace
Codegen	playwright codegen --target csharp

Playwright Test only

Test locks	Use an xUnit collection instead
Isolated retries	The runner controls the retries
UI mode	No equivalent
Speedboard	Use your own test report
Test Agents	They write TypeScript

Demo

```
npx playwright codegen --target csharp-xunit http://localhost:3000
```

The recorder writes C#, not TypeScript.

Accessibility in C#

AccessibilityTests.cs

```
var results = await Page.RunAxe(new AxeRunOptions
{
    RunOnly = new RunOnlyOptions
    {
        Type = "tag",
        Values = ["wcag2a", "wcag2aa", "wcag21a", "wcag21aa", "wcag22aa"],
    },
});

Assert.Empty(results.Violations.Select(v => v.Id).ToArray());
```

Deque.AxeCore.Playwright 4.13.0 adds RunAxe() to Page and Locator. It uses the same axe-core engine as the TypeScript package.

Same engine, same result

Both languages report the same three rules on our form: button-name, color-contrast and label. Accessibility is not a JavaScript topic.

Demo

```
aspire run
dotnet test
```

5 tests, 1 red. Compare the output with the TypeScript run.

A real browser for an agent — two ways

Both give the agent the accessibility tree, not pixels. Thus the agent writes role based locators. The server is part of Playwright since 1.62, the CLI is a separate package.

MCP server

Tools	A fixed tool schema, always in the context
Loop	The agent clicks through the page
Browser	The server starts its own

playwright-cli

Tokens	Commands only, thus no tool schema
Attach	A browser that runs, with your session
Reach	run-code puts a whole snippet in the page

Demo

```
npx playwright mcp --port 7890            # the server, no installation
npm install -g @playwright/cli@latest     # the CLI, one installation
playwright-cli attach --extension=chrome  # take the browser that is open
playwright-cli -s=chrome run-code "async page => {   # one command, not ten clicks
  await page.clock.install(); await page.reload();
  await page.getByRole('button', { name: /Deploy Now/i }).click();
  await page.clock.fastForward('00:30'); }"
```

Both drive the same browser. Only the CLI takes a browser that already runs.

Demo: an agent tests the app over MCP

The MCP server gives the agent the accessibility tree, not pixels. It is part of Playwright since 1.62.

.mcp.json

```
{
  "mcpServers": {
    "playwright": { "command": "npx", "args": ["playwright", "mcp"] }
  }
}
```

STEP 1
Prompt
Test the error path of the deploy form.

→

STEP 2
Snapshot
browser_snapshot gives the ARIA tree, with a ref for each element.

→

STEP 3
Act
browser_type and browser_click use the refs.

→

STEP 4
Result
A spec with role based locators. You review it.

playwright-cli: the agent runs code

One snippet instead of ten tool calls. The snippet has the full Playwright API.

agent-snippets/cooldown.js

```
async page => {
  await page.route('**/api/deploy', route =>
    route.fulfill({ json: { success: false, message: 'BOOM' } }));
  await page.clock.install();
  await page.reload();
  await page.getByLabel(/Who dares to deploy/i).fill('Agent');
  await page.getByRole('button', { name: /Deploy Now/i }).click();
  await page.getByText('BOOM').waitFor();
  const during = await page.locator('.deploy-button').textContent();
  await page.clock.runFor('00:30');
  const after = await page.locator('.deploy-button').textContent();
  return { during, after };
}
```

Result: {"during":"Next deploy in 30s","after":"Deploy Now 🚀 "}

```
playwright-cli open http://localhost:3000
playwright-cli run-code \
  --filename=agent-snippets/cooldown.js
```

Tokens	Commands only, no tool schema in the context
Reach	Routes, clock and more pages in one call
Repeat	The agent keeps the file and runs it again
Attach	The agent uses your open Chrome, with your login

Demo

```
playwright-cli open http://localhost:3000
playwright-cli run-code --filename=agent-snippets/cooldown.js
```

The countdown goes from 30 seconds to zero in one call.

Test Agents

`NPX PLAYWRIGHT INIT-AGENTS --LOOP=CLAUDE`

Make the agent definitions again after each Playwright update, to get the new tools.

STEP 1

Planner

Explores the app and writes a test plan in Markdown to specs/.

→

STEP 2

Generator

Makes test files from the plan and proves each locator against the running app.

→

STEP 3

Healer

Runs a failed step again, looks for the element that replaced the old one and proposes a patch.

Demo

Change the button text from `'Deploy Now'` to `'Ship It'`.

Let the healer repair the test.

FROM A REAL PROJECT

Where you must hold the reins

A generated test can look correct and be worthless

Typical defects: a hard coded wait, a CSS selector instead of a role, and an assertion that is always true.

```
// Generated — and wrong.
await page.waitForTimeout(3000);
await page.click('.p-button.p-component.deploy-button');
await expect(page.locator('body')).toBeVisible();
```

The rule that works for us

Agent	**Explores and writes the first draft.**
Human	**Reviews, then puts it in the pipeline.**

Recommendation

Let an agent write the first draft, not `playwright codegen`. The agent reads the accessibility tree and uses roles. Review each draft.

TL;DR

1. **Write few E2E tests. The test pyramid still holds.**
   An unreliable E2E test is worse than no E2E test.
2. **An E2E test with a real backend is not a frontend job.**
   Run it after each backend build too, or nightly.
3. **Make accessibility a gate on day one.**
   axe-core is a few lines in the test. The European Accessibility Act applies since June 2025.
4. **AI: let agents drive the browser with the MCP server and playwright-cli.**
   Use them to explore and to automate. A human reviews each test that they write.

Not in this talk

1.61

WebAuthn passkeys

A virtual authenticator tests a login with a passkey, with no hardware.

1.62

Component testing

A new model with stories and galleries mounts one component in a real browser.

1.48

WebSocket mocking

routeWebSocket() replaces a realtime backend, for example a live deployment log.

Also worth a look

- storageState with IndexedDB — keep a login between tests
- npx playwright trace — read a trace in the terminal, with no window
- Copy prompt in the report — give a failed test to an LLM with one click

That's all folks!

THANK YOU

Questions?

Demo project and slides available at https://github.com/vensas/playwright-2026-talk

https://www.linkedin.com/in/svenhennessen/

kontakt@vensas.de

vensas.de

Slides & Demo